# Database Lab : Live Root Blocking Lock Resolution & Kill-SOP

Duration : 60 Min.

## Scenario

As a Lead Database Administrator at NIIT, multiple customer checkout transactions are frozen due to an exclusive lock conflict on table `accounts`. You must interrogate PostgreSQL catalogs (`pg_locks` and `pg_stat_activity`), isolate the root blocking backend session, safely terminate it using `pg_terminate_backend`, and record the incident resolution audit log.

## Input Details

The environment has been pre-configured with the following local database platform instances:

- **Database Host:** `localhost` (Ubuntu Linux VM)
- **PostgreSQL Service:** `localhost:5432` | DB: `postgres` | User: `postgres` | Password: `PostgresPassword123!` | CLI: `psql`

## Username & Naming Conventions

Your candidate prefix is: **`<prefix>`** (e.g. `student-exam123`).

Resource naming: - **Lock Remediation Audit Table:** `<prefix>_lock_remediation_log`

## Task Objectives

Perform the following lock triage steps directly via `psql`:

### 1. Interrogate Active Locks & Sessions

- Connect via `psql` to `localhost:5432`.
- Query `pg_locks` and `pg_stat_activity` to inspect all active locks and identify any sessions in `idle in transaction` state.

### 2. Create Lock Remediation Audit Table

- Create table `<prefix>_lock_remediation_log` with columns:
- `incident_id SERIAL PRIMARY KEY`
- `target_engine VARCHAR(30) NOT NULL`
- `root_blocker_pid INT NOT NULL`
- `termination_command TEXT NOT NULL`
- `resolved_by VARCHAR(50) NOT NULL`
- `logged_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP`

### 3. Terminate Root Blocker & Record Audit Log

- Identify the blocker PID and execute: `SELECT pg_terminate_backend(<root_pid>);`
- Insert a record into `<prefix>_lock_remediation_log` recording the terminated PID, the exact SQL command used, and your candidate prefix.

## Verification

Once completed, run the verification script:

```
python run.py
```

The driver connects to PostgreSQL, asserts that no ungranted locks block the accounts table, and validates the audit log.

## Grading Criteria

Your performance will be evaluated based on the following test cases:

| Test Case | Requirement | Marks |
|---|---|---|
| **TC1** | Catalog Query Execution (pg_locks & pg_stat_activity) | 5 Marks |
| **TC2** | Lock Remediation Table (_lock_remediation_log) Creation | 5 Marks |
| **TC3** | Safe Backend Termination Command (pg_terminate_backend) Formulation | 5 Marks |
| **TC4** | Live Concurrency Lock Contention Resolution | 5 Marks |

**Total Score: 20 Marks**

## Important Notes

- Replace `<prefix>` with your candidate prefix.
- Never use OS kill -9 when pg_terminate_backend is available.